Il documento raccoglie chiarimenti e indicazioni per la realizzazione ed il rilascio della app Gaming Explorer, emersi durante la sessione di confronto con gli studenti di mercoledì 4 Novembre 2026.

Sommario

1. Cosa si intende esattamente con indicazione del caricamento dei dati? 2
2. Il filtro per la valutazione deve basarsi sul rating o sul Metacritic score? 3
3. Come strutturare un frontend scalabile nel tempo? 3
4. Best practice per ottimizzare la UI per l'utente? 3
5. Conta di più UX o software mantenibile? 4
6. Tutti i servizi devono avere CRUD? Friends, users, games ecc.? 4
7. Quali criteri usare per scrivere codice più chiaro? 4
8. Dobbiamo consegnare anche .env? La repo deve funzionare appena clonata? 5
9. Preferenza di lingua per commenti e documentazione? 5
10. L'uso di Postman è obbligatorio? 6
11. Occorre documentare i test effettuati con le API? 6
12. L'applicazione può girare in locale o deve essere pubblicata? 6
13. La gestione del GDPR è obbligatoria. 6

1. Cosa si intende esattamente con indicazione del caricamento dei dati?

Durante una chiamata asincrona l'utente deve capire che l'applicazione sta lavorando e che i dati non sono ancora disponibili. Ciò può accadere, per esempio, quando l'utente:

- effettua una ricerca
- applica un filtro
- apre il dettaglio di un gioco
- carica una lista di videogiochi

tra l'inizio della chiamata REST e la ricezione della risposta non dovrebbe trovarsi davanti a una schermata apparentemente bloccata o vuota.

È sufficiente mostrare un **loader, spinner, skeleton** oppure un messaggio "Caricamento...", disabilitando eventualmente le azioni che non devono essere eseguite mentre la richiesta è in corso.

L'applicazione deve comunicare chiaramente all'utente **cosa sta succedendo durante il caricamento dei dati**:

- ⏳ **Loading** → i dati stanno arrivando
- ✅ **Successo** →
  - 📋 **Dati trovati → visualizzazione dei dati**
  - 🔍 **Nessun risultato → messaggio informativo**
- ❌ **Errore** → la richiesta non è andata a buon fine e viene mostrato un messaggio
- 🔍 **Nessun risultato** → la richiesta è andata a buon fine, ma non ci sono dati da mostrare

In sintesi: **l'utente non deve mai trovarsi davanti a una schermata vuota senza capire cosa sta succedendo.**

2. Il filtro per la valutazione deve basarsi sul rating o sul Metacritic score?

Differentemente da quanto indicato nel documento di progetto, il filtro si basa sul Metacritic score e non sul rating.

3. Come strutturare un frontend scalabile nel tempo?

Occorre valutare soprattutto **modularità e separazione delle responsabilità**, che sono principi esplicitamente richiesti dal progetto.

Con Angular, per esempio, una buona soluzione potrebbe separare:

- **pages/features**, per ricerca, dettaglio, preferiti ecc.
- **components**, per elementi UI riutilizzabili
- **services**, per l'accesso alle API/backend
- **models/interfaces**, per i modelli dati
- eventuali **guards/interceptors**, se implementano autenticazione
- componenti condivisi solo dove esiste effettivamente riutilizzo

Regola pratica:

Se domani aggiungessimo Wishlist o autenticazione, non dovremmo essere costretti a riscrivere metà dell'applicazione.

4. Best practice per ottimizzare la UI per l'utente?

L'obiettivo principale è una UI semplice, intuitiva, responsive e coerente.

Occorre concentrarsi soprattutto su:

1. navigazione semplice
2. feedback immediato alle azioni
3. loading/error/no-results chiaramente visibili
4. layout responsive
5. filtri comprensibili e facilmente modificabili
6. evitare informazioni o controlli inutili
7. coerenza grafica tra le diverse pagine

Non serve cercare una UI particolarmente sofisticata: è più importante che sia facile da usare.

L'applicazione deve avere un'interfaccia intuitiva, una navigazione semplice e tempi di risposta adeguati, oltre ad adattarsi a desktop, tablet e smartphone.

5. Conta di più UX o software mantenibile?

Riportiamo la griglia di valutazione:

Criterio	Punteggio
Correttezza e completezza delle funzionalità obbligatorie	**10 punti**
Qualità del codice (organizzazione, modularità, leggibilità e buone pratiche)	**10 punti**
Interfaccia grafica, usabilità e responsive design	**10 punti**
Documentazione tecnica e repository Git	**5 punti**
Presentazione del progetto e capacità di illustrare le scelte progettuali	**5 punti**
Funzionalità Bonus	**+2 punti**

La qualità e la manutenibilità del codice, così come la UI/UX hanno lo stesso peso nella valutazione complessiva: **a entrambe le aree sono assegnati 10 punti ciascuna**.

6. Tutti i servizi devono avere CRUD? Friends, users, games ecc.?

No: non è necessario forzare un CRUD su ogni risorsa.

Gli endpoint devono riflettere i **casi d'uso dell'applicazione**. Per esempio:

- videogiochi RAWG → prevalentemente **lettura/ricerca**
- preferiti → tipicamente **aggiunta, rimozione, elenco**
- utente → operazioni necessarie all'autenticazione/profilo, se implementato
- eventuali friendship → solo le operazioni necessarie alla feature

7. Quali criteri usare per scrivere codice più chiaro?

Codice chiaro:

- nomi comprensibili
- metodi piccoli e con una responsabilità chiara
- niente duplicazioni evidenti
- separazione tra UI, logica applicativa e accesso ai dati/API
- componenti e servizi riutilizzabili quando serve

- gestione centralizzata degli aspetti trasversali, quando ragionevole
- niente codice morto
- commenti utilizzati per spiegare il **perché**, non per tradurre banalmente il codice
- struttura consistente del progetto

Non bisogna cercare l'architettura perfetta, ma occorre essere in grado di rispondere in modo esaustivo a questa domanda: «**Perché questo codice è qui e non altrove?»**

Se la risposta è chiara e la struttura è coerente, siamo sulla giusta strada.

8. Dobbiamo consegnare anche .env? La repo deve funzionare appena clonata?

Il .env reale contenente secret/API key/password non deve essere committato.

Nel file **README** bisognerà spiegare:

1. quali prerequisiti installare
2. come creare/configurare l'ambiente
3. quali variabili valorizzare
4. come avviare frontend e backend
5. eventuali inizializzazioni/migrazioni del database

La presenza o meno di tutte o parte di queste informazioni sarà oggetto di valutazione.

Non pretendiamo che git clone sia sufficiente, senza alcuna configurazione, per poter utilizzare il progetto, ma chiediamo che il **repository contenga tutto ciò che serve per capire come configurare e avviare il progetto**, senza includere credenziali o secret.

Questo è sufficiente per effettuare la valutazione del Git repository.

9. Preferenza di lingua per commenti e documentazione?

Italiano o inglese: vanno entrambe bene, l'importante è essere coerenti.

Esempio/suggerimento:

- codice e naming → inglese
- commenti tecnici → preferibilmente inglese
- README/documentazione → italiano o inglese, purché venga utilizzata una lingua in maniera consistente

Evitare qualcosa di misto del tipo:

- GetUser()
- SalvaPreferito()
- GameService
- GestioneUtentiController

10. L’uso di Postman è obbligatorio?

Postman è una piattaforma completa a supporto dello sviluppo, testing, documentazione e gestione delle API. Non è obbligatorio usarla: potete usare qualsiasi altro strumento simile.

11. Occorre documentare i test effettuati con le API?

No, non è richiesto.

12. L’applicazione può girare in locale o deve essere pubblicata?

Ai fini della challenge non è richiesta la pubblicazione dell’applicazione: può girare anche solamente in locale.

13. La gestione del GDPR è obbligatoria.

No. Può rappresentare una funzionalità bonus, se correttamente gestito e documentato.